

PROGRAMA INTEGRAL DE FORMACIÓN EN PYTHON

CURSO 2: ALGORITMOS AVANZADOS Y ESTRUCTURAS DE DATOS
(NIVEL 2 (INTERMEDIO))

Módulo 03: Recursividad y Programación Dinámica

«Las Muñecas Rusas (Matrioshkas) y la Libreta de Apuntes»

Optimización algorítmica avanzada: Caso base y recursión, desbordamiento del Call Stack,
Programación Dinámica, Memoización con `@functools.lru_cache` y Tabulación.

Instructor: William Rodríguez (Wisrovi)

Rol: AI Solutions Architect & Principal Software Engineer

Nivel del Curso: Intermedio | Python: 3.10+ | Licencia: MIT

Acerca del Autor y Mentor



William Rodríguez (Wisrovi)

AI Solutions Architect & Principal Software Engineer • Badajoz, España

Ingeniero y arquitecto de software especializado en Inteligencia Artificial Generativa, sistemas multi-agente, Visión por Computador e infraestructuras MLOps de alta disponibilidad. Creador y mantenedor de la suite de software libre **wisrovi SUITE** en PyPI con más de 26 bibliotecas enfocadas en orquestación de pipelines, caching distribuido y optimización de bases de datos.



GitHub: github.com/wisrovi



LinkedIn: [in/wisrovi-rodriguez](https://in.linkedin.com/in/wisrovi-rodriguez)



DockerHub: hub.docker.com/u/wisrovi



Website: wisrovi.dev

Metodología de Aprendizaje: La Regla de la Bicicleta 🚲

En este programa no creemos en el aprendizaje pasivo. Programar no se aprende memorizando manuales o mirando videos en segundo plano mientras tomas café; se aprende **escribiendo código**, enfrentando errores de sintaxis, depurando variables y viendo cómo responde el intérprete en tiempo real.



El Compromiso Activo del Estudiante

Abre Visual Studio Code en cada sesión. Escribe cada ejemplo con tus propias manos. Cambia los números, rompe el código deliberadamente para ver el mensaje de error de Python, y luego arréglalo. Ese proceso de experimentación es el que construye sinapsis duraderas.

Estructura Pedagógica de Cada Guía

Cada documento de esta serie está diseñado rigurosamente bajo el estándar de ingeniería de software profesional:

- **Fundamento Conceptual y Metáfora Intuitiva:** Anclaje visual del concepto abstracto a la realidad.
- **Diagrama de Flujo y Arquitectura Mental:** Representación visual de cómo fluye la información.
- **Código de Nivel Profesional:** Sintaxis limpia, comentarios línea a línea y tipado estático (PEP 484).
- **Patrones y Gotchas:** Errores comunes que cometen los desarrolladores y cómo evitarlos.

Índice General de la Sesión

Sección	Contenido y Enfoque Temático	Página
Capítulo 1	Fundamentos y Metáfora Conceptual: Pensamiento Recursivo y Subproblemas Superpuestos	Pág. 4
Capítulo 2	Diagrama de Flujo y Arquitectura: Árbol de Llamadas Recursivas vs Tabla de Memoización	Pág. 5
Capítulo 3	Implementación Práctica en Python: Fibonacci Optimizado con Memoización	Pág. 6
Capítulo 4	Patrones Avanzados, Gotchas y Debugging: Gotchas en Recursividad	Pág. 7
Cierre	Conclusiones, Notas del Mentor y Agradecimiento	Pág. 8
Anexos	Bibliografía Oficial y Enlaces de Profundización	Pág. 9

Objetivos de Aprendizaje (Competencias Clave)



Competencia Conceptual

Comprender la descomposición recursiva y cómo la memoización transforma complejidades exponenciales $O(2^n)$ en lineales $O(n)$.



Competencia Práctica

Implementar algoritmos recursivos seguros y optimizar cálculos pesados con decoradores nativos de Python.

Requisitos Previos y Entorno Recomendado

Para aprovechar al máximo esta sesión se recomienda tener instalado Python 3.10 o superior, Visual Studio Code con la extensión oficial de Python configurada, y haber completado las lecturas y ejercicios de las sesiones anteriores de la ruta formativa.

1. Pensamiento Recursivo y Subproblemas Superpuestos

La recursividad ocurre cuando una función se invoca a sí misma para resolver una versión más pequeña del mismo problema.

☀ Metáfora Central: Las Muñecas Rusas (Matrioshkas) y la Libreta de Apuntes

La recursión es como abrir una muñeca rusa (Matrioshka): abres una y hay otra idéntica más pequeña dentro, hasta llegar a la más diminuta que no se puede abrir (el Caso Base). La memoización es como tener una libreta de apuntes: cuando resuelves un cálculo difícil, anotas el resultado para no tener que volver a calcularlo jamás.

Principios Teóricos y Modelo Mental

Todo algoritmo recursivo DEBE tener al menos un Caso Base para detener las llamadas antes de saturar el Call Stack (RecursionError).

Programación Dinámica (DP): Técnica para resolver problemas complejos descomponiéndolos en subproblemas y guardando sus soluciones.

⚡ Regla de Oro en Python

Sin memoización, Fibonacci recursivo tiene complejidad $O(2^n)$; con memoización se reduce a $O(n)$.

2. Árbol de Llamadas Recursivas vs Tabla de Memoización

Eliminación de ramas redundantes en el árbol de ejecución mediante caché en memoria.



Desglose Paso a Paso del Diagrama

Fase del Flujo	Acción del Intérprete	Estado en Memoria
1. Inicialización	Llamada inicial a la función con el parámetro n .	$f(5)$ en Call Stack
2. Evaluación	Bifurcación recursiva en $f(n-1)$ y $f(n-2)$.	Subárbol de cálculos
3. Transformación	Verificación en caché: si el resultado ya existe, lo devuelve inmediatamente sin recalcular.	Hit en caché $O(1)$
4. Retorno / Salida	Si no existe, computa el caso base y almacena el resultado antes de retornar.	Guardado en memoria



Visualización Mental

La memoización es intercambiar memoria (RAM) por tiempo de CPU: un compromiso altamente beneficioso en sistemas modernos.

3. Fibonacci Optimizado con Memoización

Comparativa entre recursión ingenua y optimización con el decorador `lru_cache` de la librería estándar:

```
main.py (Python 3.10+) UTF-8 • PEP 8 Compliant

from functools import lru_cache
import time

# Versión Optimizada con Programación Dinámica
@lru_cache(maxsize=None)
def fibonacci_memo(n: int) -> int:
    if n <= 0: return 0
    if n == 1: return 1
    return fibonacci_memo(n - 1) + fibonacci_memo(n - 2)

# Cálculo instantáneo para n=100
inicio = time.perf_counter()
resultado = fibonacci_memo(100)
fin = time.perf_counter()

print(f"Fibonacci(100) = {resultado}")
print(f"Tiempo de cálculo: {(fin - inicio)*1000:.4f} ms")
```

Análisis del Código Fuente

El decorador `@lru_cache` intercepta las llamadas y almacena los resultados en una tabla hash en memoria, logrando tiempo de ejecución instantáneo.

4. Gotchas en Recursividad

Errores críticos que pueden derribar servicios productivos:

⚠ Gotcha Frecuente (Trampa de Principiante)

Olvidar el caso base o no avanzar hacia él en cada iteración, provocando un `RecursionError` por desbordamiento de pila.

Patrón Recomendado vs Antipatrón

✗ Antipatrón / Mal Código

```
def loop(n):  
    return loop(n) # RecursionError: maximum  
recursion depth exceeded
```

✓ Patrón Pythonic / Correcto

```
def loop(n):  
    if n <= 0: return 0 # Caso base  
    return n + loop(n - 1)
```

🛡 Consejo de Resiliencia en Producción

Python tiene un límite de recursión por defecto de 1000 llamadas (`sys.getrecursionlimit()`).

5. Resumen Ejecutivo y Conclusiones

Has dominado las técnicas de optimización más avanzadas de la ciencia de la computación aplicadas a Python.



Logro Alcanzado en esta Clase

Capacidad para transformar problemas intratables en algoritmos de alto rendimiento con programación dinámica.

Notas del Instructor y Sigüientes Pasos

En el Curso 3 entraremos de lleno a la Inteligencia Artificial: LLMs, Prompt Engineering, RAG y Agentes Autónomos.



Mensaje de Agradecimiento

Muchas gracias por tu entusiasmo, disciplina y dedicación al participar en este programa formativo. La programación es un superpoder que transforma vidas cuando se ejerce con constancia y curiosidad. ¡Nos vemos en la próxima sesión para seguir construyendo juntos!

«El código más elegante es aquel que cualquiera puede entender y nadie necesita reescribir.»

6. Fuentes Bibliográficas y Recursos de Estudio

Para profundizar y consolidar los conocimientos adquiridos en esta clase, consulta las siguientes referencias:

Recurso / Fuente	Descripción Temática	Enlace Oficial
Documentación Oficial de Python	Referencia canónica del lenguaje y librería estándar	docs.python.org/3/
PEP 8 - Style Guide for Python	Guía oficial de estilo, formato e indentación	peps.python.org
Real Python Tutorials	Artículos técnicos y patrones de desarrollo moderno	realpython.com
Python Type Checking (PEP 484)	Anotaciones de tipo y análisis estático en Python	docs.python.org/typing
Suite Open Source wisrovi	Paquetes Python para orquestación y alto rendimiento	github.com/wisrovi

Canales de Consulta y Comunidad

Si encuentras dificultades al replicar el código o tienes dudas sobre la arquitectura de los ejercicios, no dudes en abrir un Issue en el repositorio oficial del curso en GitHub o participar activamente en nuestras sesiones grupales de mentoría.



Desafío de Autoestudio Recomendado

Resuelve el clásico problema del cambio de monedas (Coin Change Problem) usando programación dinámica con tabulación.